University of Waterloo

Faculty of Engineering

Department of Electrical and Computer Engineering

Portable Smartphone Authentication Device

Detailed Design and Project Timeline

Group 2015.000

(names removed)

July 3rd, 2014

One of the strengths of this software project is the quantitative experimental data in Section 3.2. Analysis of experimental data counts as "quantitative technical analysis".

Contents

1 High-Level Description of Project

1.1 Motivation

A study shows that approximately half of smartphone users surveyed do not use a password or PIN to protect their phone [1]. Those who were surveyed either reported that passwords or PINs are "too cumbersome" or that they do not believe the risk to be present, certainly not enough to inconvenience themselves with a password every time they take their phone out of their pocket. Those that are forced to engage in security practices such as a PIN for a credit card often choose simple and easy to remember PINs over PINs that are hard to guess, leading to the result that 5 unique PINs can account for the PINs of around 20% of credit cards studied [2]. An unauthorized user trying to gain entry into another person's smartphone could simply guess the top 5 most frequently used PINs and have a 20% success rate. The smartphone industry has been moving to more convenient methods of securing a smartphone such as using visual patterns and facial recognition, but these can be circumvented by an unauthorized user by, for example, observing the smartphone user enter the easy to recognize visual pattern or holding up a picture of the user's face to the phone's camera, which can usually be obtained easily online. It appears that there is a lack of smartphone security mechanisms that are convenient enough to encourage their use without sacrificing the user's security. A security mechanism that can identify which users are authorized to access a smartphone without relying on an authorized user to remember a piece of information could allow for better smartphone security without inconveniencing the user.

1.2 Project Objective

The purpose of this project is to design a portable smartphone authentication device that makes mobile security more convenient and reliable. This method works by allowing users to unlock their phone without the necessity of user input and without requiring the user to enter a password or PIN every time into their smartphone. This device keeps the user's smartphone secure without inconveniencing the user and without discouraging the user to engage in smartphone security best practices. The device solves the issue of protecting the user's smartphone from unauthorized access by employing an automated authentication scheme to uniquely identify the smartphone owner. The portable authentication device allows the smartphone owner bearing the correct portable authentication device to gain access to the smartphone.

1.3 Block Diagram

Figure 1 shows a block diagram describing the high level organization of the system. The system consists of both a smartphone and the authentication device, which we shall refer to as the electronic key. The smartphone and the electronic key communicate with each other over a wireless channel.

The smartphone has a software application which functions as the master authentication service. This service looks for a nearby electronic key and issues a challenge to the key that the key must successfully solve in order to prove that it is the correct key for unlocking the smartphone. The challenge is unique such that only the correct key can solve the challenge correctly. The key itself also has an authentication service implemented in software that runs on a microcontroller. This service acts as a slave to the authentication service on the smartphone which listens for incoming challenges and sends its solution back to the smartphone. Both the smartphone and the electronic key contain a wireless communication module to support wireless communication between the two. In addition, the electronic key has a component that takes advantage of the wireless communication channel to determine the proximity to the smartphone so that it may announce its presence to the smartphone. Since the key will be a wireless device, it must be powered by a small battery. The charge level of this battery can be determined by connecting the battery to an analog input on the microcontroller in order to read its voltage level. For convenience, the key also has a status indicator to communicate to the user about events such as low battery and to indicate that it is functioning correctly. Since the key cannot accept any input from the user, the smartphone has a configuration application which allows the user to configure the key and to choose which keys are allowed to unlock the smartphone.

In the block diagram on the following page, the UI refers to a user interface that allows for an individual to interact with the phone and configure the device prior to initial use. The microcontroller has a few general purpose input/output (GPIO) pins which will be used to interface with the battery and status indicator(s).

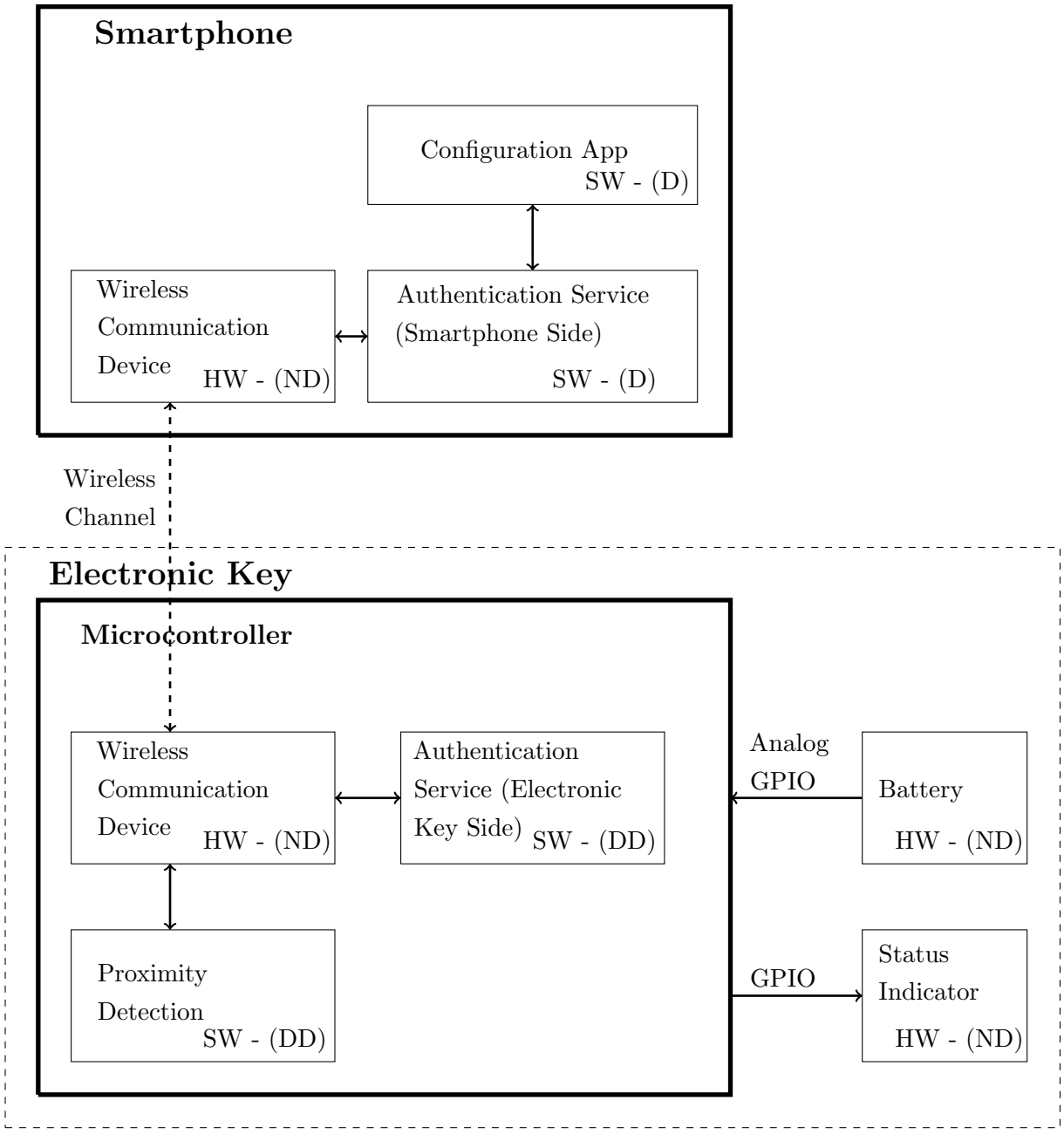

Figure 1: Block Diagram

(D) - This subsystem will be designed as a part of this project.

(ND) - This subsystem will not be designed as a part of this project.

(DD) - This subsystem will be designed with greater detail.

HW - This subsystem consists of a hardware component.

SW - This subsystem consists of a software component.

2 Project Specifications

2.1 Functional Specifications

Table 1: Functional Specifications

#	Specification	Importance
F1	The key must be placed within approximately 1.0 m from the smartphone to unlock the smartphone.	Essential
F2	The smartphone should issue a challenge to a key that attempts to unlock the smartphone that only can be solved by the correct key in order to verify that the key is the same key that the user gave permission to access the smartphone. If the key fails the challenge, the smartphone should not unlock.	Essential
F3	The phone should be unlocked within 3 seconds of the user attempting to access their phone.	Essential
F4	The key should communicate with the smartphone wirelessly.	Essential
F5	The phone should lock itself after a user-specified period of idle time (defined as the amount of time the smartphone screen is off) or after the key is placed further than approximately 1.0 m from the smartphone. Either solution is satisfactory.	Essential
F6	If the key is within range of the smartphone and does not unlock the smartphone within 3 seconds, the key should re-attempt to unlock the smartphone unless the key is known to not be permitted to unlock the phone.	Non-essential
F7	The user should be able to configure their key using the smartphone since the key will not provide a method of user input.	Essential
F8	The user should be able to select which key is allowed to unlock their smartphone.	Essential
F9	The user should be able to select multiple keys that are allowed to unlock their smartphone.	Non-essential
F10	The user should be able to change their key without acquiring new hardware. This should be supported in software.	Essential
F11	The user should have an alternative method to unlock the phone in the event that the key is not functioning. The alternative authentication method must require information that is known only to the user and must be more secure than a 15 character password.	Essential

2.2 Non-functional Specifications

Table 2: Non-functional Specifications

#	Specification	Importance
NF1	The device must fit within a 6 cm × 4 cm × 3 cm box.	Essential
NF2	The costs for developing the prototype should not exceed $500.	Non-essential
NF3	The device must be battery operated and last for at least 24 h on a single battery.	Essential
NF4	The user should be able to easily tell if the key is powered on or off.	Non-essential
NF5	The authentication method used by the system should be more secure than a 15 character password. This can be evaluated by comparing the difficulty of solving the challenge issued to the key with the difficulty of cracking a 15 character password.	Essential
NF6	The user should not be granted access to the smartphone without either the device or the alternative authentication method.	Essential
NF7	The authentication process involving the electronic key should be easier to use than entering a pin, password, or pattern to unlock the smartphone. The authentication process is easier to use if it takes less time and requires less user input.	Essential
NF8	The user should be informed if a key was recognized but is not permitted to unlock the smartphone.	Non-essential
NF9	The user should be able to check the battery level of the electronic key.	Non-essential
NF10	The user should be informed by an electronic key that it is currently being configured by a smartphone.	Non-essential
NF11	The master authentication service should run on a modern phone operating system such as Android.	Essential

3 Detailed Design

Selecting the Platform

One of the largest components of the design will be referred to as the electronic key, or simply the key. The key is a general term that refers to the hardware and software pieces that form the functional unit used for unlocking the smartphone. This preamble addresses the choice of hardware components that will support the underlying software. The first few subsections will detail the hardware functions and constraints followed by a more comparative analysis of different microcontroller solutions.

Wireless Communication

Wireless communication is one of the most important functional specifications that the design needs to satisfy, and so a considerable amount of thought was put into choosing a suitable transmission method. The four transmission methods considered were near field communication (NFC), infrared, Wi-Fi, and Bluetooth. NFC isn't a suitable choice since the Android operating system requires that the phone be already unlocked to allow for transmission. This can be bypassed by modifying (through rooting, for example) the operating system that the smartphone runs on, however this idea was discarded since it is preferable that the design works on an unmodified version of the operating system. Infrared ports are quite uncommon on smartphones, making it poor choice as a universal solution. Wi-Fi is an extremely popular choice for wireless communication, however there are issues with power consumption, reliability and range. Bluetooth radios are extremely common on modern smartphones, and transmission can be achieved at extremely low power rates. The low transfer rates of Bluetooth can become a drawback for larger data transfers, but are suitable enough for most applications. Taking all of these factors into consideration, Bluetooth communication proved to be the most suitable choice as a wireless communication method. The choice of using Bluetooth led to the process of finding a microcontroller equipped with a Bluetooth radio. Use of Bluetooth for the system satisfies functional specification F4, which states that the electronic key must communicate with the smartphone wirelessly.

Power Consumption and Battery Life

The electronic key is a completely wireless device, therefore it requires a battery power source. Battery life is an important consideration, and NF3 states that the battery should last for at least 24h on a single charge. Power consumption can be analyzed to give an estimate for approximate battery life. A chip that has a lower overall peak current is preferable. Once the chip has been chosen, a battery can be selected that is capable of satisfying the requirements of the chip. Size and capacity are also important factors that affect battery selection.

Performance, Accessibility, and Ease of Programming

The processor speed will play an important role in determining the time it takes for the authentication

scheme to encrypt and decrypt data. A faster clock speed on the processor will allow for faster decryption, allowing flexibility in the size and security of the data that is transferred and decrypted. A certain amount of random access memory (RAM) will be required as part of the decryption algorithm, along with some flash-based memory to store configuration information that is programmed by the smartphone. In addition to the power input, some general purpose input/output ports are required for the status indicator.

On the software side, its preferred that the chip be programmable in the C programming language as opposed to a proprietary language. An integrated development environment (IDE) aids in the programming and debugging of the software which helps reduce the amount of time required attempting to write the embedded software. The device should be easily programmable without requiring soldering and preferably without the required purchase of a dedicated development kit.

Comparing the Platforms

Based on the properties described above, three microcontrollers were chosen for comparison. They include the Texas Instruments CC2540, the BlueGiga BLE113, and the RFDuino RFD22301. A number of these properties are summarized in Table 3.

Table 3: Hardware Comparison of Microcontrollers [3] [4] [5]

Property	**TI CC2540**	**BG BLE113**	**RFD22301**
Power Supply (min)	2.0 V	2.0 V	1.9 V
Power Supply (max)	3.6 V	3.6 V	3.6 V
TX Peak Current	24 mA	18.2 mA	18 mA
RX Peak Current	19.6 mA	14.3 mA	18 mA
Sleep/ULP Mode Current	0.4 to 0.9 μA	0.4 μA	4 μA
Microcontroller	8051	8051	nRF51822
RAM	8 kB	8 kB	8 kB
Flash Memory	128 kB	128 kB	128 kB
CPU Frequency	16 MHz	16 MHz	16MHz

The table above shows that all three of the chips have very similar hardware specifications. Since the performance characteristics of the three alternatives are very comparable, greater emphasis was placed on the ease of accessibility for development. The CC2540 is a standalone chip requiring additional hardware and circuitry for development, increasing the amount of effort required. The BLE113 uses a proprietary programming language and requires an expensive development kit for programming and debugging making it an undesirable choice. The RFD22301 is based on the open Arduino platform, designed to make it extremely easy for developers to begin programming. The RFDuino also has an easy to use USB-based plug-and-play solution with various peripheral accessories. It will also satisfy the size specification NF1, and is low cost enough to satisfy NF2. Assuming peak transfer current occurs less than 50 % of the time, and a CR 2032 coin-cell battery has a 225 mAh rating, the RFDuino will be able to last for at least 24h to satisfy NF3. Based on these justifications, the best hardware choice is the RFDuino.

3.1 Subsystem 1: Authentication Service (Electronic Key Side)

Authentication service on the electronic key is a security mechanism which communicates with the smartphone in order to authenticate the user. In our design, the smartphone issues a challenge to the electronic key that only the correct electronic key can solve, similar to how only the correct key can unlock a deadbolt, as required by functional specification F2. The technique of digital signatures is used as the challenge: the smartphone sends data to the electronic key and the electronic key must sign the data and return the signature. Only the correct electronic key should be able to generate the correct signature. If the smartphone can verify that the signature originated from the correct electronic key, the smartphone can safely unlock itself. This sequence of events is illustrated in Figure 2.

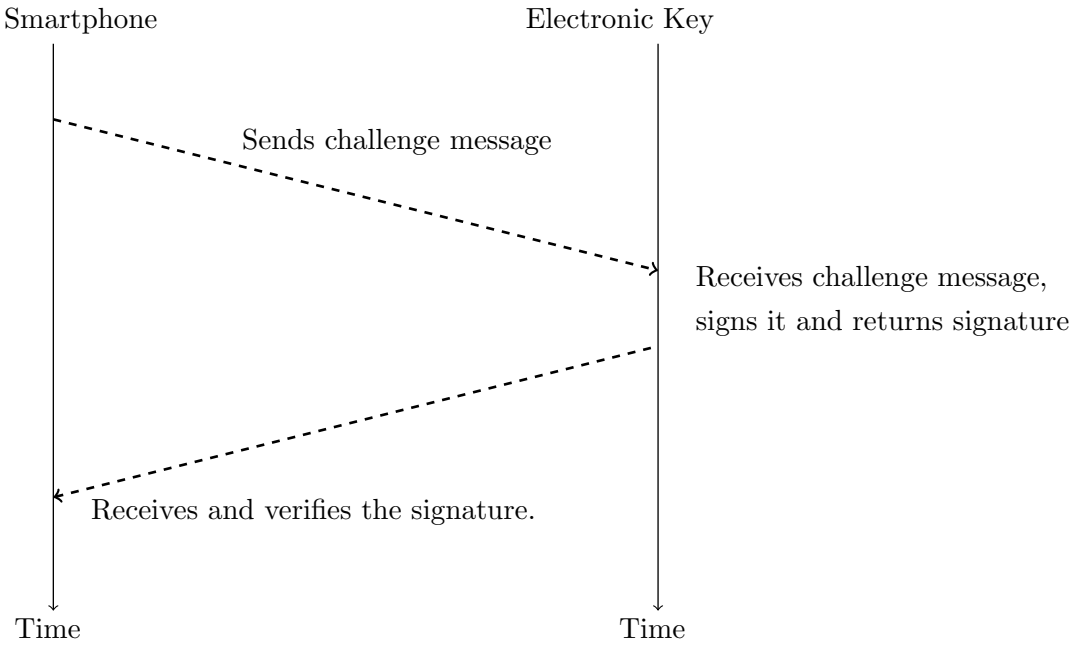

Figure 2: Signature and Verification Method

The remainder of this section will discuss the design decisions regarding the recommended authentication scheme and the implementation thereof. Additionally, the signature and verification method will also be discussed in greater detail.

3.1.1 Authentication Scheme

This section and the subsequent sections will briefly describe and analyze the authentication schemes. The schemes that will be considered are Rivest-Shamir-Adleman (RSA), Digital Signature Algorithm (DSA), and Elliptic Curve Cryptography (ECDSA). These authentication methods will be evaluated on various criteria that are the most relevant to the project. The key generation speed, memory

usage, performance, reliability and security are some of the most essential criteria that will be used to analyze the schemes.

Rivest-Shamir-Adleman (RSA)

RSA is a public key crypto-system which is implemented using modular exponentiation. The first step in the RSA scheme is the generation of two very large distinct prime numbers, chosen randomly. These numbers are then used to compute the Euler's totient function, and the product of these two numbers is used as the modulus for the public and the secret key.

This scheme is considered to be computationally secure since it is based on the Integer Factorization Problem. In order to successfully break RSA, the two prime factors should be solved. The attackers in the past have been able to break this scheme for key sizes less than 1024 bits using exhaustive search methods to determine the prime factors. A well known attack for RSA is the called General Number Field Sieve(GNFS) algorithm. GNFS serves as a threat because this algorithm is the fastest known method for factoring large integers, including 1024 bit keys. This algorithm runs in sub-exponential time. Therefore, the RSA scheme implemented using a 1024-bit key is currently not vulnerable to an attack based on GNFS.

The performance of RSA can be measured in terms of the time it takes the algorithm to generate the keys, sign and verify the messages. From Table 5, it can be seen that the time to sign a message in RSA is significantly higher than DSA and Elliptic Curve Digital Signature Algorithm (ECDSA). Similarly, the key generation time for RSA exceeds the other two authentication schemes. However, message verification on RSA is very efficient in comparison to other methods.

Digital Signature Algorithm (DSA)

In DSA authentication scheme, a digital signature is computed using a private key, a per-message secret number, the data that is to be signed, and a hash function. All of these domain parameters, along with the public key, are used to verify the identity of the sender [6].

Similar to RSA, this scheme is also considered secure. This scheme is also vulnerable to the same attack as RSA, i.e. General Number Field Sieve(GNFS) algorithm. Therefore, the time to successfully break the DSA is the same as the RSA.

Table 5 shows the performance comparison of different authentication schemes. From that table, it can be observed that the time DSA takes to sign a message is significantly less than RSA, but is very close to the time taken by ECDSA. In the case of verifying the message, DSA takes the most amount of time out of all the authentication schemes. Lastly, DSA takes the same amount of time as ECDSA to generate the key which is significantly less that the time taken by RSA.

Elliptic Curve Cryptography (ECDSA)

Elliptic Curve Cryptography is based on the computations with points on the elliptic curve. Elliptic curves are usually defined over integer modulo of a prime number ($GF(p)$, where GF is the Galois

Field and p is a prime number) or over binary polynomials ($GF(2^m)$, where m is the degree of the polynomial) [7]. ECDSA is most suitable authentication scheme for memory limited electronic devices. The key size indicates the size of the prime number or the binary polynomial in bits.

ECDSA is capable of providing the same security level as RSA and DSA but using a shorter key length [8]. Table 4 contains information about the key sizes required for various authentication schemes.

Table 4: Key sizes for Equivalent Security Levels [8]

Symmetric	**ECDSA (bits)**	**DH/DSA/RSA (bits)**
80	163	1024
128	283	3072
192	409	7680
256	571	15360

ECDSA is based on Elliptic Curve Discrete Logarithm problem. In order to break ECDSA, exponential time is required to solve the problem of elliptic curve discrete logarithm. On a quantum computer, Shor's algorithm is capable of factoring integers and computing discrete logarithms both in a run time of $O(n^3)$. Moreover, a 160 bit key can be broken on a 1000-qubit quantum computer, whereas a 2000-qubit quantum computer is required to break 1024-bit RSA [9].

ECDSA performs better in terms of the time required for signing messages, i.e. it takes the least amount of time to sign as can be seen in Table 5. The performance for ECDSA is average in comparison to RSA and DSA for message verification. The key generation time for ECDSA is comparable to DSA, which is significantly less than the time required by RSA to generate a key.

Performance Comparison of Authentication Schemes

The performance of the aforementioned authentication schemes can be measured in terms of the time it takes to generate keys, sign messages and verify messages. The table below shows the comparison between each of the schemes considered.

Table 5: Digital Signature timings (milliseconds on a 200 MHz Pentium Pro) [7]

	RSA ($e = 3$)	**DSA - 1024**	**ECDSA - 168**
Sign	43	7	5
Verify	0.6	27	19
Key Generation	1100	7	7

As mentioned earlier, the smartphone and the electronic key will use the method of digital signatures. The properties of interest from the table above are the times required to sign the challenge and verify the solution. The best performance in terms of signing is the ECDSA scheme which takes the least amount of time, and the worst scheme is RSA. On the contrary, RSA takes the least amount

of verification time. In this scenario, DSA is the worst performing scheme as it takes a significantly large amount of time to verify a message.

Selected Authentication Scheme

In order to decide the authentication scheme suitable for this project, various criteria depending on their importance were selected. The decision matrix in Table 6 shows the criteria used for evaluating different authentication schemes. Each criterion is assigned a weight based on its importance and each scheme is assigned a score on the scale of 1 - 10, where 1 represents the lowest score and 10 represents the highest score. The scores assigned to each scheme are based on the discussion of authentication schemes above.

Table 6: Comparison between RSA, DSA, and ECDSA

Criteria	**Weight**	**RSA**	**DSA**	**ECDSA**
Security	0.40	8	8	7
Speed	0.30	6	7	8
Memory Usage	0.20	5	5	8
Reliability	0.10	9	9	9
Total	1	6.9	7.7	**9.2**

The ECDSA authentication scheme has the best score in comparison to the other two schemes. Therefore, the schemes that will be used to implement the authentication service on the electronic key is ECDSA. ECDSA is a variation of Digital Signature Algorithm (DSA) which is based on elliptic curve cryptography.

3.1.2 Certificate Generation

The certificates are randomly generated and require either a cryptographically secure random number generator (RNG) or a pseudo-random number generator (PRNG) to ensure the security of generated certificates. This section will briefly discuss random number generation on Android smartphones and RFDuino microcontrollers. The section will also focus on the method of transferring the certificate from the one device to the other.

RNG on Android

Android smartphones use Java's `SecureRandom` class which by default generates a seed (an array of bytes used to bootstrap random number generation) from an internal entropy source, such as `/dev/urandom` [10]. The RNG gathers environmental noise from the device and other sources into an entropy pool [11]. The Android devices have sufficient source of entropy. The main sources of entropy on Android are from interrupt requests, disk events, and user input. In fact, the entropy contributed by disk events alone ensures `/dev/urandom` is properly seeded [12].

RNG on RFDuino

The RFDuino microcontroller consists of a Nordic semiconductor, $nRF51822$. The RNG on this semiconductor generates a true non-deterministic random numbers based on internal thermal noise. The random numbers generated from these kinds of RNGs are not vulnerable to cryptographic attacks. Unlike RNG on Android, the RNG on this semiconductor does not require a seed value [13]. However, in order to make use of this built in RNG, the development kit will be required which will incur an additional cost. As an alternative an RNG could be designed and implemented but it will take more resources to develop.

Discussion

Our decision to implement certificate generation on either the smartphone or the electronic key is based on three important factors: RNG security, memory required, and certificate generation time.

Based on the discussion about random number generation above, the RNG on Android uses `/dev/urandom` for generating random values which is capable of generating pseudo random numbers. However, the RNG on the microcontroller would have to be implemented using thermal noise to generate true random numbers. Since the motivation behind this project is to provide security for users' information on smartphones, choosing a device that guarantees true randomness proper is critical.

The minimum number of bits required to implement a secure ECDSA scheme is 163 bits. The RFDuino microcontroller has a memory of 8 kB, which is capable of supporting this scheme. The smartphone has much more memory than the microcontroller and is able to support ECDSA. Additionally, the certificate generation time on the smartphone is much faster than the electronic key. The reason for this is that the smartphone is equipped with high performance processors in comparison to the RFDuino microcontroller.

Table 7 is a decision matrix chart which is used to determine whether the certificate should be generated on the smartphone or the electronic key. The weights to criteria are assigned based on their importance. The scores are on the scale of 1 to 10, where 1 is the lowest score and 10 is the highest.

Table 7: Decision Matrix to determine Key Generation

Criteria	**Weight**	**Smartphone**	**Electronic Key**
Security	0.5	9	9
Speed	0.2	7	5
Memory Usage	0.1	7	7
Resources Required	0.2	9	3
Total	1.0	8.4	6.8

From the table above, it can be concluded that the certificate should be generated on the smartphone, as it is faster than the electronic key. Certificate generation is a one time process which will take place when the user first configures the electronic key on the smartphone, so the time taken by

certificate generation is not as important as the security of the system. Moreover, since there is no built in RNG on the microcontroller additional resources will be required to implement one.

Transferring Certificate to Electronic Key

This section will briefly describe certificate generation and the process by which it will be transferred to the electronic key. The certificate comprises of a public key and a secret key. Since the certificate is generated on the smartphone, the secret key needs to be transfered to the electronic key. The electronic key requires the secret key because it will use the it along with the hash of the challenge to sign the challenge. The signed data will then be sent back to the smartphone for verification.

For ECSDA, a public key comprises of a tuple that only needs to be publicly available. After generating the public key the smartphone can store a copy of it. In order to transfer the certificate over to the electronic key, the Diffie-Hellman (DH) key exchange protocol will be used. By using DH, an encrypted channel can be setup between the smartphone and the electronic key. The channel is setup so that information can be securely sent from the smartphone to the electronic key.

3.1.3 Implementation on the Microcontroller

Based on the recommendations to use ECDSA above to authenticate the electronic key, the electronic key needs to support the ECDSA signature algorithm. This can be facilitated using the Micro-ECC library that implements ECDSA for microcontrollers with limited program memory and RAM available, making it suitable for this system [14]. This library is also resistant against side-channel attacks. It was also discussed that the electronic key needs to support the Diffie-Hellman key exchange algorithm in order to set up an encrypted channel for installing the ECDSA digital certificate on the electronic key. Diffie-Hellman key exchange is available in the OpenSSL library [15]. These two libraries need to be installed on the electronic key.

To satisfy functional specification F6 which states that the key should re-attempt to unlock the smartphone every three seconds until it succeeds or is informed by the smartphone that it is not authorized, the electronic key will wait for a success or failure message from the smartphone and repeat the authentication process if a message is not received from the smartphone in three seconds since the last authentication attempt. If a failure message is sent by the smartphone the electronic key should light up its indicator LED in red for three seconds to inform the user of the failure, satisfying non-functional specification NF8.

3.2 Subsystem 2: Proximity Detection

3.2.1 Locking and Unlocking Protocol

Choosing when to lock and unlock the phone can have an effect on system usability and battery life. There are two protocols we consider when deciding when to lock and unlock the phone.

Method 1

The first method considered specifies that the phone should be unlocked when the electronic key comes into the range of the smartphone and that the phone should remain unlocked while the electronic key is within range. Once the electronic key goes out of range of the phone, the phone should lock and present the user with the alternate authentication method. This satisfies specification F1 which states that the electronic key must be within 1.0 m to unlock the phone, specification F5 which states that the phone should lock itself automatically when the electronic key goes out of range, and specification F11 which states that the user should be presented with an alternate phone unlocking method if the electronic key is not working (in the case of this approach, the alternate unlock method will be presented whenever the phone is locked).

This method has the advantage that the phone is already unlocked when the user wishes to access their phone and, as a side effect, satisfies specification F3 which states that the phone should be unlocked within 3 seconds of the user attempting to access their phone. The disadvantage of this approach is that it either requires the phone to poll the key periodically to determine if it is within range or it requires the electronic key to periodically announce its presence. Both approaches negatively affect battery life of the electronic key as the periodic presence polling/announcing requires energy to transmit and receive the required information. To ensure the system is responsive in a reasonable amount of time while maintaining an acceptable battery life, the presence polling/announcing should be performed every 5 seconds, in between which the electronic key may remain in a low power state.

Care must be taken if the polling approach is used to mitigate denial of service attacks as a malicious user could continuously poll another user's electronic key in order to drain its battery. These attacks can be prevented by allowing the electronic key to wait for 5 seconds before responding to a smartphone polling for its presence. Furthermore, there is additional power consumption when using the polling approach as the electronic key must remain out of low power mode so that it can receive polling messages from a nearby smartphone. For these reasons, the announcing approach is preferred.

While the smartphone is unlocked, there is potential for another electronic key to spoof the electronic key that unlocked the phone to come within range of the phone and keep the smartphone unlocked. To prevent this, the smartphone and electronic key should pair over Bluetooth before the smartphone is unlocked to ensure the smartphone is continuously connected to the same electronic key that unlocked it.

Method 2

The second method considered specifies that the electronic key should attempt to unlock the smartphone when the user turns on the screen of the smartphone and that the smartphone should lock itself after the screen is turned off for a period of time specified by the user. When the smartphone screen turns on, the smartphone should test if the electronic key is within the permitted range of the smartphone and allow the electronic key to unlock the smartphone if it is within range. This satisfies specification F1 which states that the electronic key must be placed within 1.0 m of the smartphone to unlock the smartphone, and specification F5 which states that the smartphone should lock itself after a period of idle time. If the smartphone can unlock using this method within 3 seconds of the user turning on the smartphone screen, this method would additionally satisfy specification F3.

This method requires the electronic key to listen for the smartphone to announce that its screen can turn on, which has the same battery life issues as the polling approach in Method 1; the electronic key must remain out of low power mode.

Protocol Specification

Given the analysis presented above, Method 1 using the announcing approach is better suited for this project. This method has the best battery life as it allows the electronic key to remain in a low power mode for most of the time it is powered. It also has the best unlocking time as the phone is already unlocked whenever the user wishes to access their phone. The final locking and unlocking protocol is as follows:

1. The electronic key must announce its presence every 5 seconds. From this announcement, the smartphone must use the signal strength value to determine the distance between the electronic key and itself.

2. If the electronic key is within 1.0 m of the smartphone and the smartphone is locked, the electronic key and smartphone must pair over Bluetooth. The smartphone must then issue a challenge as specified in Section 3.1 and unlock itself if the challenge is solved by the electronic key. If the electronic key fails the challenge, the paired connection is dropped. The smartphone should remain unlocked while the electronic key is within 1.0 m of the smartphone.

3. If the electronic key is beyond 1.0 m from the smartphone and the smartphone is unlocked, the smartphone should lock itself.

4. If the smartphone does not receive a presence announcement from the electronic key within 15 seconds of a previous presence announcement (allowing enough time for two missed announcements), the smartphone should assume that the electronic key is not within range and lock itself.

3.2.2 Presence Announcement

The Bluetooth protocol has an advertising mode whereby the Bluetooth device advertises its presence to other Bluetooth devices periodically [16]. The period of these advertisements can be chosen manually. On the RFduino, the Bluetooth protocol stack issues an interrupt request when this period is over and the device needs to advertise its presence. These interrupt requests can wake the RFduino from low power state allowing the RFduino to remain in a low power state in between presence advertisements. This advertisement mode is sufficient for announcing the presence of the electronic key to nearby smartphones as required by the Locking and Unlocking protocol specified above. The Bluetooth module in the electronic key should remain in advertisement mode while not keeping a smartphone in the unlocked state. While the electronic key is paired to the smartphone over Bluetooth, presence does not need to be announced.

3.2.3 Measuring Proximity

To determine the distance between the smartphone and the electronic key, we are measuring the power in the Bluetooth radio signal with the received signal strength indicator (RSSI). RSSI measurements can be measured anytime that a signal is received on a device, removing the requirement of having to send a separate message just to measure the signal strength. RSSI and distance are related by the inverse square law, meaning that doubling the distance will reduce the measured power by a few decibels. RSSI is affected by a number of environmental factors, such as humidity and proximity to other objects.

There are a number of different methods that can be used to determine which RSSI value will be used in approximating distance between the smartphone and the electronic key. The first approach, is the most naive method through which simple polling records the RSSI value at every possible time, without performing any calculations or transformations to the values. This naive polling method has the advantage of being extremely quick in producing a value, however is more prone to inaccuracy due to fluctuating and sporadic data. An alternative approach uses an averaging algorithm that sums the result of several measurements and divides by the total number of iterations. As the number of iterations increases, the average settles to a more consistent value. This technique is more likely to obtain an accurate value, but takes a longer period of time. A more sophisticated approach calculates a moving average using a recursive filter. Impulses that are added into the system will take an infinite amount of time to completely die out, resulting in a more accurate result. The moving average also removes the timing requirement of having to run for a certain number of iterations before deciding upon a value, making it the most ideal choice.

An experiment was carried out to obtain a relationship between distance and RSSI values. The RFDuino was fixed to a location and a program was loaded onto the microcontroller that periodically broadcasting a message using the maximum signal strength. A third party application from Nordic

Semiconductors was installed onto the smartphone, which allows for real-time measurements of RSSI values. The smartphone was initially placed directly next to the RFDuino and given a few seconds to calibrate. The RSSI value was then recorded, and the smartphone was moved to a further distance. This experiment was repeated a number of times, in a few different scenarios. The first scenario assumed the smartphone and the RFDuino were placed in a direct line of sight from one another, without an obstructions. The second scenario was performed with the RFDuino in hand, simulating when the device might be worn on the wrist of the user. The final scenario was conducted with the smartphone placed in the pocket of the user. A number of different distances were tested from extremely close proximity to about 1.6 m. Figure 3, shown below, details the results from these experiments.

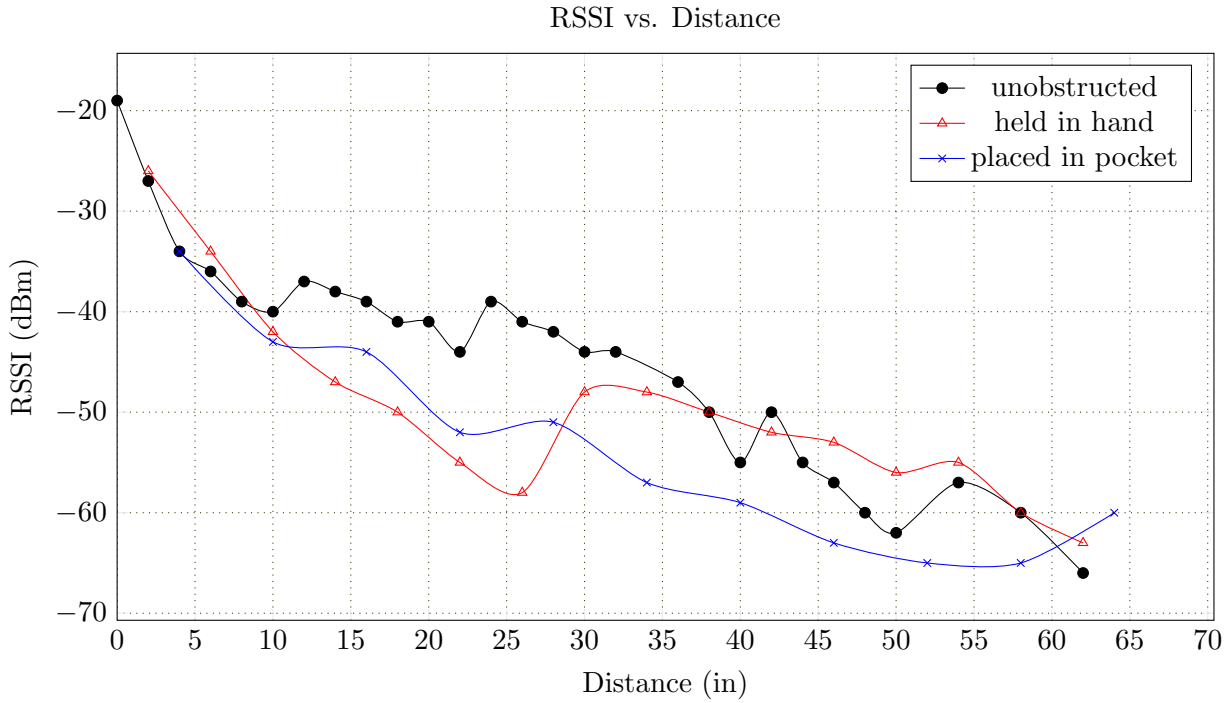

Figure 3: Experimental Data of Measured RSSI vs. Distance

The RSSI values need to be able to differentiate between distances that range from 0 to more than 1.0 m in order to satisfy specifications F1 and F5. To satisfy these requirements, certain thresholds will need to be specified that correspond to specific distances. In all of the tested scenarios, the reported RSSI values were above -45 dBm for distances less than 0.25 m. At distances of 1.0 m or more, the RSSI value falls to -60 dBm or below. Periodic polling of the RSSI value can be used to lock the smartphone when values of less than -60 dBm are recorded, indicating that the electronic key has moved far enough away from the smartphone. A sampling of data will be required to ensure that the device has moved out of range for a certain period of time to prevent a poor signal strength reading from locking the device even when in range.

3.3 Subsystem 3: Authentication Service (Smartphone Side)

3.3.1 Implementing Cryptographic Scheme

To ensure the identity of the electronic key, the smartphone will pose challenges to the electronic key in the manner described under Protocol Specification in Section 3.1. The authentication service is implemented on an Android phone, satisfying non-functional specification NF11 which states that the master authentication service should run on a modern phone operating system such as Android. After the smartphone receives a solved challenge from the key it sends a message back to the key indicating either success or failure. This success/failure is determined by using the ECDSA public key stored on the phone to verify that the solved challenge sent by the electronic key. If the electronic key does not receive either a success or failure message within 3 seconds of sending the signed challenge, it retransmits the data to the smartphone. This meets F6 which states that if the key is within range of the smartphone and does not unlock the smartphone within 3 seconds, the key should re-attempt to unlock the smartphone unless the key is known to not be permitted to unlock the phone.

3.3.2 Certificate Generation

Based on the discussion of certificate generation in Section 3.1.2, it was decided that the public and secret keys will be generated on the smartphone. After which, the secret key will transfered to the electronic key using Diffie-Hellman key exchange protocol. The public key will only be stored on the smartphone.

3.3.3 Alternate Authentication Method

The electronic key is the primary method used for unlocking the smartphone since it is much easier than entering a pin, password, or pattern to unlock the smartphone, and it takes less time and doesn't require any input from user, satisfying NF7. However this is not always the case, for example when the electronic key is not within proximity or if the key is lost, stolen, or damaged. In order to account for these situations, an alternative authentication method will be implemented, satisfying NF6 which states that the user should not be granted access to the smartphone without either the device or the alternative authentication method. The default lock screen with enable/disable capability will be used for this alternative approach. The default lock screen will be disabled when the electronic key is in range and will be enabled when it is not. When the lock screen is enabled, it will require the user to enter the password which was set at the time of electronic key configuration. The configuration app will require the user to set a password that is at least 15 alpha-numeric characters which satisfies NF5. This method ensures that the user always has access to the information on the phone with an added security benefit.

3.4 Subsystem 4: Configuration App

The configuration app is a software component that is installed on the Android smartphone. This application is required to setup and manage electronic keys and configure other parameters of the system. Specification F7 states that the user should be able to configure their electronic key using the smartphone, since the key does not allow for user input. An option to configure multiple electronic keys could also be provided, however this has been classified as a non-essential specification in F9.

To satisfy these requirements, the configuration application should be able to scan for near-by unconfigured electronic keys. It should also be able to connect to and configure an unconfigured key to ready it for use with the smartphone that the configuration app is running on. The user interface on the application should provide a list of all available near-by unconfigured electronic keys. The user should be able to select one of the electronic keys in the list to connect to and, upon connecting, the application should automatically generate and install a cryptographic key pair on the electronic key using the process described in Section 3.1.2, satisfying functional specification F7. The newly configured electronic key should then be added to a list of electronic keys that have been configured for the user's smartphone and the user should be shown this list after configuration is complete. From this list of electronic keys that have been configured for this smartphone, the user should be able to reconfigure any of the electronic keys or remove any of the electronic keys so that the electronic key will no longer be able to unlock the user's smartphone. Any electronic key in the list of electronic keys configured for use with the user's smartphone should be able to unlock the user's smartphone, satisfying functional specification F8 and F9.

An option to install a new cryptographic key pair on a particular electronic key that is already configured for the user's smartphone should also be provided. Just as a user should be able to change their password on any password protected system, a user should be able to change the cryptographic key pair on their electronic key. This option satisfies functional specification F10.

While the electronic key is being configured by the application, the application should signal the electronic key to flash its indicator LED to indicate to the user that the electronic key is successfully being configured. This satisfies non-functional specification NF10.

The following additional options should be provided by the configuration app for the user's convenience:

- An option to select the relative distance at which an electronic key should unlock the phone (the default value should be approximately 1.0 m). This setting should affect all electronic keys configured for the user's smartphone.
- An option to select the relative distance at which an electronic key should lock the phone (the default value should be approximately 1.0 m).
- An option to identify a particular electronic key. The user might have several electronic keys

configured for their smartphone. This option would send a message to the electronic key and the electronic key would respond by flashing its indicator LED briefly to identify itself. This would help the user identify which electronic key is which, and would also help the user identify whether a particular electronic key is powered on or not, satisfying non-functional specification NF4.

- An option to check the battery life of a particular electronic key. This satisfies functional specification NF9.

Figure 4, shown below, details the design of the configuration application. The image on the left shows the main settings screen, with a few different options. On this screen, the user can choose to list all keys, which will present the user with the screen shown on the right. The settings page gives options to set values for locking and unlocking distance, which can be entered in the dialog box that appears when clicking on the list item. The screen shown on the right lists all of the currently configured electronic keys for that smartphone. The name, Bluetooth MAC address, public key fingerprint, and RSSI are shown for each configured device. A list of devices that aren't currently configured are listed below, along with the name, Bluetooth MAC address, and RSSI.

Figure 4: Configuration Application Mock-up for Android

4 Discussion and Project Timeline

4.1 Discussion on Design

Hours Completed

Table 8 indicates the total number of hours that the group has completed. The approximate number of hours per week is slightly less than the expected 10 hours, and so each group member will need to contribute approximately 15 hours per week for the remainder of the term while working on the prototype construction. These estimates seem reasonable, now that the design phase has been completed.

Table 8: Total Hours Completed to Date

Group Member	Hours Completed
(names removed)	59.25
	61.5
	63.5
	62

Engineering Knowledge

The entire authentication scheme is based on the principles of cryptography, which is discussed in great detail in the Computer Security course, ECE 458. Digital signatures and public cryptography (e.g. RSA) are some of the authentication schemes which were covered in the lectures, and the content was used to form a large part for our design. Concepts from ECE 455, Embedded Software, were also used in planning out the design for the microcontroller. A series of lectures in this course focused on side-channel attacks on embedded devices, which is one of the major concerns for our project.

Design Evaluation

Yes, our design satisfies all essential and non-essential specifications. Our design considers all of the specifications, however it is not certain whether or not our actual implementation will be able to meet these expectations. There are a few non-essential specifications that would be nice to see in a final product, but they may not be necessary for the construction of our final prototype. If we finish the prototype with some time to spare, we could improve upon some of the non-essential specifications, which will make our implementation closer to a final product.

The design is elegant in that it considers security as the primary concern. We aren't aware of any solutions that are solving the problem in the same way that we are. Our device is a standalone component that could be compatible on a number of different smartphones. The design solves a problem that people are going to become more and more concerned with in the future.

Most of the design has been thought out quite thoroughly, so we don't expect many refinements

before starting to build the prototype. There will likely be some difficulties that arise once we start writing the software, but these shouldn't affect any major design components. Some minor changes to our design components may be required to accommodate these difficulties.

Safety Hazards

The initial prototype will be built using a development kit, and so it does not contain any circuitry that we have designed. A more refined prototype will be built that will have some of our own circuitry, and so we will need to consider the safety during this stage. This device will need to be enclosed in a plastic box, which will also protect potentially sharp edges such as the pins of the microcontroller. The design will also require some sort of battery, and so a potential hazard exists should we opt to utilize a lithium-ion battery.

4.2 Project Timeline

Figure 5, shown below, provides an approximate timeline for the project. The timeline is categorized into the two courses, ECE 498A and ECE 498B. The months of August and April are excluded, along with the co-op term between the two academic terms. The shaded region indicates the current progress.

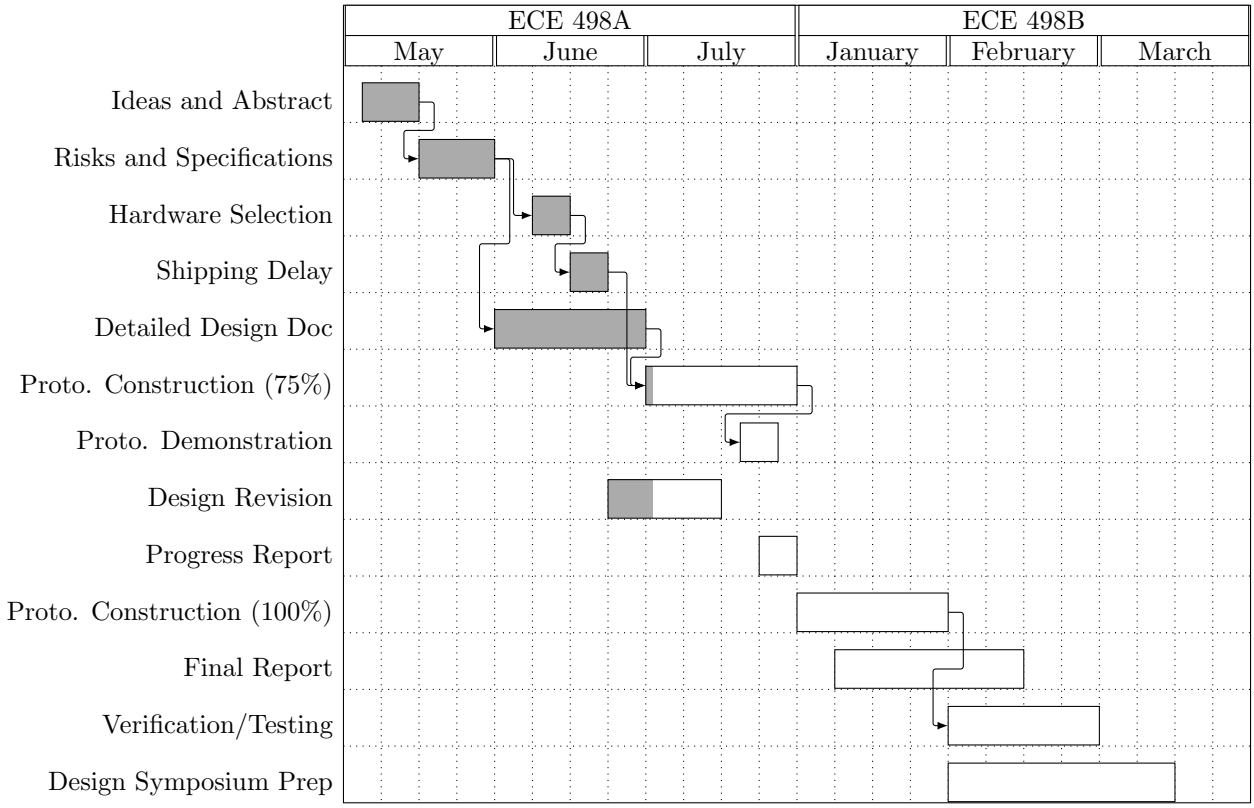

Figure 5: Project Timeline for ECE 498A and ECE 498B

We have just started on constructing the prototype, with hopes of having it about 75% completed by the end of 498A. During the month of July, we will start work on the two separate platforms independently; namely the Android application and electronic key software. This will also easily allow different group members to work concurrently, allowing us to accomplish more tasks in a shorter period of time. We've done some initial experimentation with measuring RSSI values and distance, and can use some of this analysis for designing our authentication service.

For 498B, we've allocated at least a month to finish our prototype construction. This time is flexible, since we wanted to make sure to give adequate time and resources for the final report and design symposium. We've also added a month for verification and testing which we can use flexibly if we're still working on our prototype. This time can also be used to stress-test our system to ensure it's security.

References

[1] "Survey Shows Smartphone Users Choose Convenience Over Security," http://confidenttechnologies.com/news_events/survey-shows-smartphone-users-choose-convenience-over-security, [Jun 2, 2014], (no author named).

[2] "PIN Analytics," http://www.datagenetics.com/blog/september32012/, 2009 – 2013, [Jun 2, 2014], (no author named).

[3] "BLE113 Bluetooth Smart Module," https://www.bluegiga.com/en-US/products/bluetooth-4.0-modules/ble113-bluetooth--smart-module/, 2013, [Jun 10, 2014].

[4] "CC2540 2.4ghz Bluetooth Low Energy System-on-Chip Solution," http://www.ti.com/product/cc2540/, 2014, [Jun 10, 2014].

[5] "RFD22301 RFDuino BLE SMT," http://www.rfduino.com/product/rfd22301-rfduino-ble-smt/, 2014, [Jun 10, 2014].

[6] P. D. Gallagher, "Digital Signature Standard (DSS)," http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf, p. 15, July 2013, [Jun 29, 2014].

[7] M. J. Weiner, "Performance Comparison of Public-Key Cryptosystems," *CryptoBytes*, vol. 4, no. 1, pp. 3–4, 1998, [Jun 29, 2014].

[8] K. Lauter, "The Advantages of Elliptic Curve Cryptography of Wireless Security," http://msr-waypoint.com/en-us/um/people/klauter/ieeefinal.pdf, February 2004, [Jun 29, 2014].

[9] C. P. Williams, *Explorations in Quantum Computing*. Springer Publishing Company, 2011, http://books.google.ca/books?id=QE8S--WjIFwC&pg=PA287&lpg=PA287&dq=breaking+ECC&source=bl&ots=BJCLSPXA-O&sig=ylMZq3B2NNbhCgkWNja46qZtYTA&hl=en&sa=X&ei=u9qxU4n-JtGPqgbr5YGQDg&ved=0CGIQ6AEwCQ#v=onepage&q=breaking%20ECC&f=false.

[10] "SecureRandom," http://developer.android.com/reference/java/security/SecureRandom.html, June 2014, [Jun 30, 2014], (no author named).

[11] "RANDOM(4) - Linux Programmer's Manual," http://man7.org/linux/man-pages/man4/random.4.html, March 2013, [Jun 30, 2014], (no author named).

[12] Y. Ding, "Android Low Entropy Demystified," Internet: http://accepted.100871.net/jni.pdf, [Jul 2, 2014].

[13] "nRF51822 - Multiprotocol Bluetooth 4.0 low energy/ 2.4 GHz RF SoC," http://www.100y.com.tw/pdf_file/39-Nordic-NRF51822.pdf, p. 23, 2013, [Jun 30, 2014], (no author named).

[14] K. Mackay, "Micro-ecc," Internet: http://kmackay.ca/micro-ecc/, June 2014, [Jul 2, 2014].

[15] "Openssl: Documents dh(3)," Internet: https://www.openssl.org/docs/crypto/dh.html, [Jul 2, 2014].

[16] M. Galeev, "Bluetooth 4.0: An introduction to bluetooth low energy-part i," Internet: http://www.eetimes.com/document.asp?doc_id=1278927, July 2011, [Jul 2, 2014].